ERHVERVSAKADEMI KØBENHAVN

# Week 2 — Data Filtering

Simple + Complex WHERE Clauses (MariaDB + DBeaver)
Comparison operators, ranges, lists, NULL, LIKE, AND/OR precedence

# Session Plan

Setup check + recap
Comparisons
Exercise 1
BETWEEN / ranges
Exercise 2
Break
IN / NOT IN + NULL basics
Exercise 3
LIKE patterns
Exercise 4
AND/OR precedence + parentheses
Final challenge + wrap

EK ERHVERVSAKADEMI KØBENHAVN

# Book Mapping — SQL in easy steps

- **Primary reading:**
- Chapter 7 — Simple data filtering (comparisons, ranges, non-matches, NULL)
- Chapter 8 — Complex data filtering (IN/OR alternatives, evaluation order, LIKE)

- **Helpful refreshers (if needed):**
- Chapter 5 — Retrieving data from tables (SELECT basics)
- Chapter 6 — Sorting retrieved data (ORDER BY)

# Environment — MariaDB + Docker + DBeaver

- **Setup check (5 minutes):**
- Container running? (docker ps)
- DBeaver connected? (host, port, user, password)
- Run: SELECT VERSION();
- You can see schemas in the navigator

- **If your connection is missing:**
- In DBeaver: Database > New Connection > MariaDB
- Common Docker defaults: host=localhost, port=3306, user=root
- Tip: if MariaDB is mapped to another port, use that port instead

# Dataset — Sample data (evu_shop) 1/3

- **Run once in DBeaver SQL Editor:**
- CREATE DATABASE IF NOT EXISTS evu_shop;

- CREATE TABLE customers (
- customer_id INT PRIMARY KEY,
- full_name VARCHAR(80) NOT NULL,
- city VARCHAR(40) NOT NULL,
- email VARCHAR(120) NULL,
- created_at DATE NOT NULL
- );
- CREATE TABLE products (
- product_id INT PRIMARY KEY,
- product_name VARCHAR(80) NOT NULL,
- category VARCHAR(40) NOT NULL,

# Dataset — Sample data (evu_shop) 2/3

- CREATE TABLE orders (
- order_id INT PRIMARY KEY,
- customer_id INT NOT NULL,
- product_id INT NOT NULL,
- status VARCHAR(20) NOT NULL,
- total DECIMAL(10,2) NOT NULL,
- ordered_at DATETIME NOT NULL,
- shipped_at DATETIME NULL,
- FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
- FOREIGN KEY (product_id) REFERENCES products(product_id)
- );
-
-

# Dataset — Sample data (evu_shop) 3/3

- INSERT INTO products VALUES
- (10,'SSD 1TB','Hardware',899.00),
- (11,'Keyboard (Mechanical)','Hardware',699.00),
- (12,'SQL Book','Books',249.00),
- (13,'Docker Course','Training',1199.00),
- (14,'Mouse','Hardware',299.00);

- INSERT INTO orders VALUES
- (1001,1,12,'PAID',249.00,'2024-04-03 10:05','2024-04-04 09:30'),
- (1002,2,10,'PAID',899.00,'2024-04-05 14:10',NULL),
- (1003,3,14,'CANCELLED',299.00,'2024-04-07 08:02',NULL),
- (1004,4,13,'PAID',1199.00,'2024-04-10 16:45','2024-04-12 12:10'),
- (1005,5,11,'PENDING',699.00,'2024-04-12 11:20',NULL),
- (1006,6,10,'PAID',899.00,'2024-04-15 18:05','2024-04-16 10:00');

# Warm-up Recap — SELECT + WHERE is Row Filtering

- SELECT chooses columns (projection) — what you show
- WHERE chooses rows (selection) — what you keep
- A good filter is: correct, readable, and index-friendly
- Strategy when stuck: start broad, then tighten the WHERE one condition at a time

# Comparison Operators  (Ch. 7)

- Use WHERE with comparisons: =, <>, !=, <, <=, >, >=
- Compare numbers, dates, and strings — but be explicit with quotes
- For 'not equal', prefer <> (standard) or != (common)
- Tip: when checking dates/times, compare using DATE/DATETIME columns, not strings

# Comparisons — Examples  (Ch. 7)

- -- Customers in Aarhus
- SELECT customer_id, full_name, city
- FROM customers
- WHERE city = 'Aarhus';

- -- Orders above 700
- SELECT order_id, status, total
- FROM orders
- WHERE total >= 700;

- -- Not cancelled
- SELECT order_id, status
- FROM orders
- WHERE status <> 'CANCELLED';

# Exercise 1 — Basic Comparisons (18 min)

1. List all customers created after 2024-02-01
2. List orders with total between 250 and 900 using >= and <=
3. Find orders that are NOT status = PAID
Stretch: show customer_id and email where email is not NULL

**Schema: evu_shop**

# Ranges with BETWEEN  (Ch. 7)

- BETWEEN is inclusive: min and max are included
- Works for numbers and dates; be careful with DATETIME ranges
- For end-exclusive ranges (recommended with time): use >= start AND < end
- Readable alternative: total BETWEEN 250 AND 900

# BETWEEN — Examples  (Ch. 7)

- -- Inclusive range
- SELECT order_id, total
- FROM orders
- WHERE total BETWEEN 250 AND 900;

- -- Safer pattern for DATETIME windows (end-exclusive)
- SELECT order_id, ordered_at
- FROM orders
- WHERE ordered_at >= '2024-04-05 00:00'
-  AND ordered_at <  '2024-04-12 00:00';

# Exercise 2 — Ranges (18 min)

1. Find orders with total BETWEEN 700 AND 1200
2. Find orders placed from 2024-04-05 up to (not including) 2024-04-12
3. Predict: what happens with BETWEEN for DATETIME without a time part?

**Schema: evu_shop**

# Lists with IN and NOT IN  (Ch. 8)

- Use IN to match 'one of these values': status IN ('PAID','PENDING')
- NOT IN is useful, but NULL can surprise you (3-valued logic)
- Keep your lists short; for many values, use a lookup table

# IN — Examples  (Ch. 8)

- -- Match one of several statuses
- SELECT order_id, status, total
- FROM orders
- WHERE status IN ('PAID', 'PENDING');

- -- Combine with range
- SELECT order_id, status, total
- FROM orders
- WHERE status IN ('PAID', 'PENDING')
-  AND total >= 700;

# Exercise 3 — Lists with IN (18 min)

1. Show orders where product_id IN (10, 11, 14)
2. Show orders where status NOT IN ('CANCELLED','PENDING')
Stretch: join orders to products, filter by category IN ('Hardware','Books')

**Schema: evu_shop**

# NULL — Missing/Unknown Value  (Ch. 7)

- NULL is not 'empty' and not 0 — it means missing/unknown
- Comparisons with NULL (e.g., email = NULL) do not behave like normal equality
- Use IS NULL and IS NOT NULL predicates
- Think 3-valued logic: TRUE / FALSE / UNKNOWN

# NULL Checks — Examples  (Ch. 7)

- -- Customers missing an email
- SELECT customer_id, full_name, email
- FROM customers
- WHERE email IS NULL;

- -- Orders not shipped yet
- SELECT order_id, status, shipped_at
- FROM orders
- WHERE shipped_at IS NULL;

# Exercise 4 — NULL Handling (15 min)

1. Find customers with email IS NULL
2. Find orders where shipped_at IS NOT NULL
3. Discussion: why does 'email = NULL' not work?

**Schema: evu_shop**

# Pattern Matching with LIKE  (Ch. 8)

- LIKE uses wildcards: % (many chars) and _ (one char)
- Example: city LIKE 'Aa%' matches Aarhus and Aalborg
- Escape % or _ when you need them literally (ESCAPE or backslash)
- Case sensitivity depends on collation — don't assume!

# LIKE — Examples  (Ch. 8)

- -- Names starting with 'A'
- SELECT customer_id, full_name
- FROM customers
- WHERE full_name LIKE 'A%';

- -- Cities starting with 'Aa'
- SELECT customer_id, city
- FROM customers
- WHERE city LIKE 'Aa%';

- -- Escaping a literal underscore
- -- WHERE some_col LIKE 'abc_%' ESCAPE '\';

# Exercise 5 — LIKE Patterns (18 min)

1. Find customers whose full_name contains 'a' (any position)
2. Find products where product_name starts with 'Key'
Stretch: customers in cities starting with 'Aa' AND email IS NOT NULL

**Schema: evu_shop**

# AND / OR / NOT — Precedence Matters  (Ch. 8)

- SQL evaluates operators in a defined order (precedence)
- In MariaDB: NOT binds tighter than AND, AND binds tighter than OR
- Rule of thumb: use parentheses whenever you mix AND and OR
- Make intent obvious — your future self is the main audience

# Precedence — Always Parenthesize  (Ch. 8)

- -- Without parentheses: AND evaluated before OR
- SELECT order_id, status, total FROM orders
- WHERE status = 'PAID'
-  OR status = 'PENDING'
-  AND total >= 700;

- -- Clear intent: (PAID or PENDING) and expensive
- SELECT order_id, status, total FROM orders
- WHERE (status = 'PAID' OR status = 'PENDING')
-  AND total >= 700;

# Exercise 6 — Precedence Puzzles (15 min)

Predict results, then run each query:
1) status = 'PAID' OR status = 'PENDING' AND total >= 700
2) (status = 'PAID' OR status = 'PENDING') AND total >= 700
Write a 1-sentence rule explaining the difference

**Schema: evu_shop**

# Putting It Together — Complex Filters  (Ch. 8)

- Combine tools: IN + BETWEEN + LIKE + IS NULL
- Prefer readable queries: line breaks + consistent parentheses
- Validate step-by-step: start with FROM, then add WHERE parts
- When using NOT IN, think about NULLs in the data

# Complex Filter — Worked Example  (Ch. 8)

- SELECT o.order_id, c.full_name, p.category, o.status, o.total, o.shipped_at
- FROM orders o
- JOIN customers c ON c.customer_id = o.customer_id
- JOIN products  p ON p.product_id  = o.product_id
- WHERE p.category IN ('Hardware', 'Books')
-   AND o.total BETWEEN 250 AND 1200
-   AND (c.city LIKE 'Aa%' OR c.city = 'Copenhagen')
-   AND o.shipped_at IS NULL
- ORDER BY o.total DESC;

# Exercise 7 — Final Challenge (20 min)

Create ONE query that answers:
'Show PAID or PENDING orders, total >= 700, not shipped yet, customer has email'
Return: order_id, customer name, status, total, ordered_at, shipped_at
Add ORDER BY total DESC
Stretch: also allow customers in cities starting with 'Aa' OR in Copenhagen

**Schema: eyu_shop**

ERHVERVSAKADEMI KØBENHAVN

# Wrap-up — Key Takeaways